



COLLEGE FEEDBACK CLASSIFIER

Submitted by: CHINTA LAKSHMI NIVAS JASWANTH

23BCE9554

College: VIT – AP University

Branch: Computer Science and Engineering

Date of Submission: 23/06/2025

Table of Contents:

S.No	Title	Page.no
1	Introduction	3
2	Objective	3
3	Tools & Technologies Used	4
4	Methodology/Working	5-6
5	Code Snippets with Explanation	7-13
6	Output Screenshots with Explanation	14-18
7	Link of The Project	19
8	Challenges Faced & Solutions	19-20
9	Conclusion	20
10	References	21

Introduction:

The College Feedback Classifier is an AI-powered solution designed to help educational institutions better understand and organize student feedback. Students often provide open-ended feedback that includes concerns or suggestions related to academics, campus facilities, or administrative services. Manually reading and sorting this feedback is time-consuming and prone to oversight.

This project leverages IBM Watsonx.ai Foundation Models (FLAN-T5) and few-shot prompting techniques to automatically classify open-ended feedback into one of three categories: Academics, Facilities, or Administration. The classified feedback allows institutions to make data-driven decisions and prioritize improvements in specific areas, improving the overall student experience.

Objective:

The primary objective of this project is to develop an automated system that can classify open-ended student feedback into predefined categories such as Academics, Facilities, and Administration.

Specifically, the project aims to:

- 1) Analyze free-form textual feedback submitted by students.
- 2) Categorize each feedback entry based on its theme or topic.
- 3) Use few-shot prompting with IBM watsonx.ai Foundation Models to perform multi-class classification without traditional machine learning training.
- 4) Provide institutions with structured, actionable insights to support better decision-making and service improvements.

Tools & Technologies Used:

The following tools, libraries, and platforms were used to develop and deploy the College Feedback Classifier:

Development Environment / IDEs

- 1) IBM Watson Studio – Cloud-based Jupyter notebook environment for data science and AI workflows.
- 2) Jupyter Notebook – Used for writing and executing Python code interactively.

Libraries and Frameworks

- 1) pandas – For data manipulation and CSV file handling.
- 2) ibm_boto3 – IBM's SDK for accessing Cloud Object Storage.
- 3) boto3 – Low-level configuration interface used with ibm_boto3.
- 4) ibm-watson-machine-learning – SDK to interact with IBM Watsonx Foundation Models.

Platforms and Services

- 1) IBM Cloud Object Storage (COS) – Used to upload and access the feedback dataset in CSV format.
- 2) IBM Watson Machine Learning (WML) – Used to run foundation model-based inference.
- 3) IBM watsonx.ai – Core platform enabling access to large language models like FLAN-T5.

Methodology / Working:

The project follows a structured pipeline to classify open-ended student feedback into predefined categories using IBM Watsonx.ai foundation models and prompt engineering.

Step-by-Step Workflow:

- 1)Dataset Collection
- 2)Synthetic student feedback data was created and stored as a CSV file (college_feedback_expanded.csv).
- 3)Each row contains feedback text and its manually labeled category.
- 4)Data Upload to IBM Cloud
- 5)The CSV file was uploaded to IBM Cloud Object Storage (COS) in a dedicated bucket.
- 6)Notebook Setup in Watson Studio
- 7)A Jupyter notebook was created inside IBM Watson Studio.
- 8)Required libraries were installed and initialized.
- 9)Data Retrieval
- 10)The notebook connects to IBM COS and loads the feedback data into a Pandas DataFrame using ibm_boto3.
- 11)Foundation Model Configuration
- 12)IBM FLAN-T5 (XXL) foundation model was selected using the ibm-watson-machine-learning SDK.
- 13)Generation parameters were configured (e.g., max tokens, decoding strategy).
- 14)Few-Shot Prompting Setup
- 15)A few-shot prompt template was defined with 3 labeled feedback examples to guide the model.

For each feedback line in the dataset:

17)The feedback is appended to the few-shot prompt.

18)The full prompt is sent to the FLAN-T5 model.

19)The predicted category is stored.

Results Generation:

20)Predictions are added to the original DataFrame.

21)The final results are exported as a new CSV file:
college_feedback_with_predictions.csv.

Flowchart:

[Student Feedback CSV]



[Upload to IBM COS]



[Watson Studio Notebook]



[Load Data with ibm_boto3]



[Define Few-Shot Prompt Template]



[Loop: Append Feedback → Generate Category with FLAN-T5]



[Store Predictions in DataFrame]



[Export Results as CSV]

Code Snippets with Explanation:

1) `!pip install -U ibm-watson-machine-learning`

Explanation:

This command installs or upgrades the IBM Watson Machine Learning SDK, which is required to access foundation models like FLAN-T5 or Mistral through the IBM Watsonx platform within the Jupyter notebook environment.

2) `import getpass`

```
wml_credentials = {  
    "url": "https://us-south.ml.cloud.ibm.com",  
    "apikey": getpass.getpass("Enter your IBM Cloud API Key: ")  
}
```

Explanation:

This code securely sets up the Watson Machine Learning (WML) credentials needed to access IBM watsonx.ai services. It uses `getpass.getpass()` to safely input the API key without exposing it in the notebook, ensuring security when running the model.

3) `import os`

`try:`

```
    project_id = os.environ["PROJECT_ID"]
```

`except KeyError:`

```
    project_id = input("Enter your Watson Studio Project ID: ")
```

Explanation: This block retrieves the Project ID from the environment variables in Watson Studio.

```

4) import ibm_boto3
from botocore.client import Config
import types
import pandas as pd
def _iter_(self): return 0
cos_client = ibm_boto3.client(
    service_name='s3',
    ibm_api_key_id='QxCyWvbyiaf7ZlCstYDwbSO2k88J27qyby12NWabhzm',
    ibm_auth_endpoint="https://iam.cloud.ibm.com/oidc/token",
    config=Config(signature_version='oauth'),
    endpoint_url="https://s3.us-south.cloud-object-storage.appdomain.cloud"
)
bucket = 'bucket-p73ncjho1bgvuyh'
object_key = 'college_feedback_expanded.csv'
body = cos_client.get_object(Bucket=bucket, Key=object_key)['Body']
if not hasattr(body, "_iter_"): body._iter_ = types.MethodType(_iter_, body)
df = pd.read_csv(body)
df.head()

```

Explanation:

This block connects to IBM Cloud Object Storage (COS) and loads the feedback dataset into a Pandas DataFrame. `ibm_boto3` is used to create a COS client using your API key and authentication endpoint. The bucket and `object_key` identify the exact file (`college_feedback_expanded.csv`) stored in your cloud storage.


```
5) from ibm_watson_machine_learning.foundation_models import
    Model

from ibm_watson_machine_learning.foundation_models.utils.enums
import ModelTypes

from ibm_watson_machine_learning.metanames import
    GenTextParamsMetaNames as GenParams
```

Explanation:

This code imports the necessary components from the IBM Watson Machine Learning SDK to use foundation models such as FLAN-T5 or Mistral.

Model: The main class used to interact with the foundation model.

ModelTypes: Contains the list of available model types (e.g., FLAN_T5_XXL).

GenParams: Allows you to configure generation parameters like maximum tokens, decoding method, etc.

These imports are essential for setting up and using large language models in Watsonx.ai.

```
6) model_id = ModelTypes.FLAN_T5_XXL

parameters = {
    GenParams.MAX_NEW_TOKENS: 10,
    GenParams.DECODING_METHOD: "greedy"
}

model = Model(
    model_id=model_id,
```

```
params=parameters,  
    credentials=wml_credentials,  
    project_id=project_id  
)  
model.get_details()
```

Explanation:

This block initializes the FLAN-T5 XXL foundation model from IBM Watsonx.ai for classification tasks.

ModelTypes.FLAN_T5_XXL: Specifies the pre-trained model to use.

GenParams: Sets parameters for the model's response generation:

MAX_NEW_TOKENS = 10: Limits the length of output (just the label).

DECODING_METHOD = "greedy": Ensures consistent, deterministic output.

The Model object is then created using the selected model, your credentials, and your Watson Studio project ID.

Finally, model.get_details() retrieves metadata about the model instance.

7) few_shot_prompt = ""

Classify the following student feedback into one of these categories: Academics, Facilities, or Administration.

Example 1:

Feedback: "The computer labs do not have updated software."

Category: Facilities

Example 2:

Feedback: "Professors are often not available after class."

Category: Academics

Example 3:

Feedback: "It takes too long to get fee receipts processed."

Category: Administration

""

Explanation:

This is the few-shot prompt template used to guide the foundation model. It includes three labeled examples that demonstrate how to classify feedback into one of the three categories:

1.Academics

2.Facilities

3.Administration

When the actual student feedback is added to this prompt, the model uses these examples as a reference to determine the correct category. This technique helps the model perform accurate classification without traditional training, relying purely on language understanding.

8) results = []

feedbacks = df["Feedback"].tolist()

for fb in feedbacks:

 prompt = f"{few_shot_prompt}\nFeedback: \"{fb}\" \nCategory:"

 response = model.generate(prompt=prompt)

 results.append(response['results'][0]['generated_text'])

Explanation:

This loop processes each feedback entry and generates a predicted category using the few-shot prompt and the FLAN-T5 model.

Each feedback (fb) is inserted into the prompt structure.

`model.generate(prompt=...)` sends the prompt to the foundation model, which returns a classification.

The result (e.g., "Academics", "Facilities", or "Administration") is appended to the results list.

This is the core prediction step of the project, using few-shot prompting for multi-class classification.

```
9) df["Predicted_Category"] = results
df[["Feedback", "Predicted_Category"]].head()
```

Explanation:

This step adds the predicted categories (generated by the model) to the original DataFrame `df` as a new column called "Predicted_Category".

It then displays the first few rows of the DataFrame, showing:

- 1.The original student Feedback
- 2.The corresponding Predicted_Category assigned by the model

This allows for quick visual validation of how well the model has categorized the feedback.

```
10)
df.to_csv("/home/wsuser/work/college_feedback_with_predictions.csv",
index=False)
```

Explanation:

This line exports the final DataFrame — which includes the original feedback and the model's predicted categories — into a CSV file named `college_feedback_with_predictions.csv`.

It saves the file in the `/home/wsuser/work/` directory inside Watson Studio, making it accessible for download or further use.

`index=False` ensures that row numbers are not written into the CSV file.

This is the final step in the workflow, producing a structured and shareable output file.

11) with

```
open('/home/wsuser/work/college_feedback_with_predictions.csv', 'r')  
as f:
```

```
    content = f.read()
```

```
print(content)
```

Explanation:

This block opens and reads the contents of the exported predictions CSV file directly within the notebook.

It uses Python's built-in `open()` function to read the file as plain text.

`print(content)` displays the full file contents, which is helpful for verifying the output or copying it manually if needed.

This step is especially useful when the file browser is not available or direct download links are not working.

Output Screenshots with Explanation:

1)

```
Requirement already satisfied: ibm-watson-machine-learning in /opt/conda/envs/Python-RT24.1/lib/pytho
Requirement already satisfied: requests in /opt/conda/envs/Python-RT24.1/lib/python3.11/site-packages
Requirement already satisfied: urllib3 in /opt/conda/envs/Python-RT24.1/lib/python3.11/site-packages
Requirement already satisfied: pandas<2.2.0,>=0.24.2 in /opt/conda/envs/Python-RT24.1/lib/python3.11/
Requirement already satisfied: certifi in /opt/conda/envs/Python-RT24.1/lib/python3.11/site-packages
Requirement already satisfied: lomond in /opt/conda/envs/Python-RT24.1/lib/python3.11/site-packages (
Requirement already satisfied: tabulate in /opt/conda/envs/Python-RT24.1/lib/python3.11/site-packages
```

Explanation:

This cell output shows that the notebook environment already contains the IBM Watson Machine Learning SDK (ibm-watson-machine-learning) and related packages such as:

- 1.requests – For making HTTP requests to the API
- 2.pandas, numpy – For data manipulation and analysis
- 3.ibm-cos-sdk – For interacting with IBM Cloud Object Storage
- 4.importlib-metadata, packaging – For package compatibility
- 5.lomond, tabulate – Supporting components

The output includes the exact versions of each installed dependency, ensuring compatibility with IBM Watson services.

2)

	Feedback	True_Category
0	The classrooms are too small and the fans don't...	Facilities
1	The professors explain concepts very clearly.	Academics
2	The admin office delays issuing important docu...	Administration
3	There are not enough projectors in the seminar...	Facilities
4	The timetable is not well-structured.	Academics

Explanation:

This table displays the original dataset, where each row contains:

A student feedback statement

Its corresponding manually assigned category (ground truth), such as:

1.Facilities

2.Academics

3.Administration

This information serves as the reference or benchmark for evaluating the model's predictions. It is essential in:

1.Validating model performance

2.Comparing predicted results with actual categories (for accuracy calculation)

3.Demonstrating the classification task

 This screenshot is useful to show what kind of data the model was given and what the expected (true) outcomes were.

3)

```
[{"model_id": "google/flan-t5-xxl",
  "label": "flan-t5-xxl-11b",
  "provider": "Google",
  "source": "Hugging Face",
  "functions": [{"id": "text_generation"}],
  "short_description": "flan-t5-xxl is an 11 billion parameter model based on the Flan-T5 family.",
  "long_description": "flan-t5-xxl (11B) is an 11 billion parameter model based on the Flan-T5 family. It is a supervised model trained on a variety of supervised tasks converted into a text-to-text format, and fine-tuned on the Fine-tuned Language Network (FLAN) dataset. The model is designed to perform a wide range of natural language processing tasks, including question answering, summarization, and text generation.",
  "terms_url": "https://huggingface.co/google/flan-t5-xxl/blob/main/README.md",
  "input_tier": "class_2",
  "output_tier": "class_2",
  "number_params": "11b",
  "min_shot_size": 0,
  "task_ids": ["question_answering",
    "summarization",
    "retrieval_augmented_generation",
    "classification",
    "generation"]}]
```

Explanation:

This output provides detailed metadata about the foundation model used in the project: FLAN-T5 XXL, retrieved via `model.get_details()`.

Key highlights from the output:

- 1) **Model ID:** google/flan-t5-xxl
- 2) **Description:** A large language model (11 billion parameters) fine-tuned with instruction-following capabilities for zero-shot and few-shot tasks.
- 3) **Model Type:** Encoder-decoder, capable of generating text and performing classification.
- 4) **Source:** Hosted by Hugging Face, developed by Google.
- 5) **Supported Tasks:** Classification, QA, summarization, extraction, generation.
- 6) **Limits:** Can process input sequences up to 4096 tokens with 4095 output tokens.
- 7) **Lifecycle Info:** Available since July 2023, to be deprecated in May 2025 and withdrawn by July 2025.

4)

	Feedback	Predicted_Category
0	The classrooms are too small and the fans don't...	Facilities
1	The professors explain concepts very clearly.	Academics
2	The admin office delays issuing important docu...	Administration
3	There are not enough projectors in the seminar...	Facilities
4	The timetable is not well-structured.	Academics

Explanation:

This output displays the model-generated predictions for each student feedback entry. It shows two columns:

- 1)Feedback – The open-ended input provided by students.
- 2)Predicted_Category – The category assigned by the FLAN-T5 model using few-shot prompting.

The categories include:

- 1)Facilities
- 2)Academics
- 3)Administration

This confirms that the model was able to:

- 1)Interpret the feedback correctly
- 2)Classify each entry into the relevant theme
- 3)Generate meaningful and structured output

 This screenshot is the core result of the project and directly demonstrates the success of the AI classification process.

5)

	A	B	C	D	E	F
1	Feedback	True_Cate	Predicted_Category			
2	The classro	Facilities	Facilities			
3	The profes	Academics	Academics			
4	The admin	Administr	Administration			
5	There are i	Facilities	Facilities			
6	The timet	Academics	Academics			
7	The Wi-Fi	Facilities	Facilities			
8	The exam	Administr	Administration			
9	The library	Academics	Facilities			
10	Too much	Administr	Administration			
11	The faculty	Academics	Academics			
12	The cafete	Facilities	Facilities			
13	The intern	Academics	Academics			
14	There is no	Facilities	Facilities			
15	The online	Administr	Administration			
16	Labs do no	Facilities	Facilities			
17	The faculty	Academics	Academics			
18	There is no	Administr	Administration			
19	The air cor	Facilities	Facilities			
20	The curric	Academics	Academics			
21	Administr	Administr	Administration			
22	Exam hall	Facilities	Facilities			
23	Clean drin	Facilities	Facilities			
24	There are	Facilities	Facilities			
25	The library	Academics	Facilities			
26	Most facul	Academics	Academics			
27						

Explanation:

This is the complete classification output exported to a CSV file named college_feedback_with_predictions.csv. It contains three columns:

- 1)**Feedback:** The original open-ended student comment.
- 2)**True_Category:** The correct category manually assigned for evaluation.
- 3)**Predicted_Category:** The category predicted by the FLAN-T5 foundation model using few-shot prompting.

This file:

- 1)Demonstrates that the model successfully classified the majority of feedback entries correctly.
- 2)Provides a direct comparison between true and predicted labels.
- 3)Enables the calculation of accuracy or evaluation metrics (e.g., precision, recall) if needed.

Link of the Project:

https://dataplatform.cloud.ibm.com/analytics/notebooks/v2/3780ff9c-075f-4665-8b37-c7d7012ec6d2/view?access_token=bccdec281115d956235a6a2b6ef9625a7e27edfc575630eded9f68dfe3145c92&context=wx

Challenges faced and Solutions:

1. API Key Security

Challenge: Accidentally exposed the IBM Cloud API key while uploading the notebook to GitHub.

Solution: Revoked the old key immediately, created a new secure API key, and used ``getpass.getpass()`` to prevent future exposure.

2. Model Access Error (`403` or not associated with WML instance)

Challenge: Faced a `403` error stating that the project ID was not associated with a Watson Machine Learning (WML) instance.

Solution: Linked the correct WML instance to the project through IBM Cloud Resource Configuration and ensured the proper ``project_id`` was used.

3. File Download Issues in IBM Cloud

Challenge: The exported CSV was generated but not directly downloadable using ``FileLink()`` or browser file viewer.

Solution: Manually listed the working directory using ``os.listdir()``, verified the file, and downloaded it via the Watson Studio file browser panel.

4. Model Output Parsing

Challenge: Encountered an error like `TypeError: string indices must be integers` when accessing model output.

Solution: Adjusted the code to ensure the response from `model.generate()` was treated as a dictionary and parsed correctly using:
`response['results'][0]['generated_text']`

5. Classifier Output Length & Token Limit

Challenge: Sometimes the model generated longer outputs or included additional text.

Solution: Used generation parameters like `MAX_NEW_TOKENS = 10` and `DECODING_METHOD = "greedy"` to keep outputs short and deterministic.

Conclusion:

The College Feedback Classifier project successfully demonstrated how IBM Watsonx.ai foundation models can be used to automatically categorize open-ended student feedback into relevant themes such as Academics, Facilities, and Administration using few-shot prompting.

By leveraging the FLAN-T5 model, the system was able to understand natural language feedback and generate accurate classifications without traditional training. This approach simplifies the process of analyzing large volumes of textual feedback and enables educational institutions to make data-driven decisions and focus on specific areas of improvement.

The project also highlighted the efficiency and effectiveness of foundation model-based classification in real-world applications and reinforced the importance of prompt engineering, structured output, and cloud integration.

References:

IBM watsonx.ai Documentation

<https://www.ibm.com/docs/en/watsonx>

IBM Watson Machine Learning SDK

<https://pypi.org/project/ibm-watson-machine-learning/>

FLAN-T5 Model (Google / Hugging Face)

<https://huggingface.co/google/flan-t5-xxl>

IBM Cloud Object Storage Documentation

<https://cloud.ibm.com/docs/cloud-object-storage>

Pandas Documentation

<https://pandas.pydata.org/docs/>

Few-Shot Prompting Overview (Papers with Code)

<https://paperswithcode.com/method/few-shot-learning>

Jupyter Notebook (Project Jupyter)

<https://jupyter.org/>